

Project 1 Proposal: Centrality Measures in the GitHub Social Network

DATA-620, Web Analytics, Fall 2026

Supervised by: Professor Alain Ledon

Submitted by: Mehreen Ali Gillani

Dataset: SNAP/MUSAE GitHub Social Network

Source: <https://snap.stanford.edu/data/github-social.html>

Citation: Rozemberczki, Allen & Sarkar (2019), *Multi-scale Attributed Node Embedding*, arXiv:1909.13021

1. Dataset Selection

I have identified the **SNAP/MUSAE GitHub Social Network** dataset, which meets the assignment requirements:

- **Network data:** An undirected graph of mutual-follow relationships between GitHub users (~289,003 edges, ~37,700 nodes).
- **Categorical variable:** Each node is labeled with a binary ml_target attribute: 0 = Web developer, 1 = ML developer.

The dataset consists of:

- **musae_git_edges.csv** — edge list (id_1, id_2)
- **musae_git_target.csv** — node metadata (id, name, ml_target)
- **musae_git_features.json** — node features (optional; not required for centrality analysis)

This dataset is well-suited for comparing centrality measures across categorical groups because it is large, real-world, and has a heavily skewed degree distribution (mean degree ≈ 15.3 , max $\approx 9,458$), making group comparisons meaningful.

2. High-Level Plan

Step 1 — Load the data:

Read the edge list and target files into pandas DataFrames using `pd.read_csv()`.

Step 2 — Build the graph and attach labels:

Construct an undirected networkx graph using `nx.from_pandas_edgelist()`, then attach each node's `ml_target` label as a node attribute for group comparisons.

Step 3 — Compute the two required centrality measures:

- **Degree centrality** via `nx.degree_centrality(G)`: measures how well-connected each developer is.
- **Eigenvector centrality** via `nx.eigenvector_centrality(G, max_iter=1000)`: measures how connected a developer is to other well-connected developers (i.e., influence).

Store both scores in a DataFrame indexed by node ID, along with the `ml_target` label.

Step 4 — Compare across categorical groups:

Split centrality scores by `ml_target` into Web vs. ML developer groups. Compute summary statistics (mean, median, standard deviation) and visualize distributions with side-by-side box plots.

Step 5 — Statistical testing:

Run a Welch's **t-test** to compare mean centrality between the two groups. If normality is violated (likely, given the skewed distribution), use the Mann-Whitney U test instead. Report the test statistics, p-value, and effect size.

Step 6 — Interpretation:

Determine whether degree and/or eigenvector centrality differ significantly between Web and ML developers, and discuss what this implies about the structure of each community.

3. Hypothetical Outcome and Prediction

Research question:

Do Web developers and ML developers differ in degree centrality and eigenvector centrality within the GitHub mutual-follow network?

Hypothetical outcome:

- **Degree centrality:** Web developers are expected to have significantly higher mean degree centrality than ML developers, since the web development community on GitHub is larger and more broadly interconnected. Prediction: $H_1: \mu_{\text{degree}}(\text{Web}) > \mu_{\text{degree}}(\text{ML})$, supported by a significant t-test ($p < 0.05$) and positive Cohen's d.
- **Eigenvector centrality:** ML developers may show comparable or higher eigenvector centrality despite lower degree centrality, if they cluster around a few highly influential ML hubs (e.g., popular framework maintainers). Prediction: $H_1: \mu_{\text{eigenvector}}(\text{Web}) \neq \mu_{\text{eigenvector}}(\text{ML})$, potentially with ML developers over-represented in the high-eigenvector tail.

What this would demonstrate:

If the t-test confirms a significant difference in degree centrality between groups, it would show that **degree centrality can serve as a predictive signal for a developer's domain** (Web vs. ML). If eigenvector centrality shows a different pattern, it would highlight that the two communities are structured differently, one broad and diffuse (Web), the other concentrated around influential hubs (ML).

4. Scope and Deliverables

This proposal: Identifies the dataset, describes the loading plan, and states a hypothetical prediction. No analysis is performed.

Project 1: Will include full implementation: data loading, graph construction, computation of degree and eigenvector centrality, statistical testing across categorical groups, visualizations, and discussion of predictive implications.

5. References

- Rozemberczki, B., Allen, C., & Sarkar, R. (2019). Multi-scale Attributed Node Embedding. *arXiv:1909.13021*.
- SNAP/MUSAE GitHub Social Network: <https://snap.stanford.edu/data/github-social.html>